

PROYECTO FINAL**ÍNDICE**

Contenido	1
Estructura del Proyecto	2
Capas Principales	3
Archivos Importantes	3
Seguimiento del Proyecto	5
Resultados del Proyecto	6
Diagramas	6
Pruebas Realizadas	8
Capturas	9
Resultados Finales	12

CONTENIDO

El proyecto final contiene los siguientes temas vistos en el curso:

- Arquitectura Hexagonal y principios de Arquitectura Limpia.
- Diseño de un servicio de órdenes.
- Separación en capas: dominio, aplicación, infraestructura y API.
- Entidades, estados y reglas de negocio.
- Casos de uso y puertos.
- Adaptadores de entrada y salida.
- API REST con FastAPI.
- Validación mediante esquemas Pydantic.
- Protección mediante API key.
- Persistencia con SQLAlchemy y SQLite.
- Migraciones de base de datos con Alembic.
- Pruebas unitarias, de contrato, integración y E2E.
- Cobertura de código.
- Lint y comprobación estática de tipos.
- Auditoría de dependencias.
- Construcción de imágenes Docker multistage.
- Ejecución del contenedor con usuario no root.

- Automatización mediante GitHub Actions.
- Construcción del paquete y de la imagen Docker en CI.
- Documentación técnica y diagramas del sistema.
- Evidencias del funcionamiento de la aplicación.

ESTRUCTURA DEL PROYECTO

El proyecto se encuentra dentro del repositorio principal `curso-python/final-orders/`, y el workflow de CI/CD en `curso-python/.github/workflows/final-orders-ci.yml`. La estructura principal es:

```
final-orders/
├── .dockerignore
├── .env.example
├── Dockerfile
├── docker-entrypoint.sh
├── README.md
├── alembic.ini
├── poetry.lock
├── pyproject.toml
├── docs/
│   ├── diagrams/
│   └── evidence/
├── migrations/
│   ├── env.py
│   ├── script.py.mako
│   └── versions/
│       └── <migración_inicial>.py
├── src/
│   ├── __init__.py
│   └── orders_api/
│       ├── __init__.py
│       ├── main.py
│       ├── api/
│       ├── application/
│       ├── domain/
│       └── infrastructure/
└── tests/
    ├── conftest.py
    ├── contract/
    ├── e2e/
    ├── integration/
    └── unit/
```

CAPAS PRINCIPALES

- **domain/**. Contiene las *entidades*, estados, catálogo y *reglas de negocio*.
- **application/**. Contiene los *casos de uso*, comandos y *puertos*.
- **infrastructure/**. Contiene la configuración, la conexión con la base de datos, los *modelos* SQLAlchemy y el *repositorio*.
- **api/**. Contiene los *endpoints* FastAPI, los esquemas HTTP y las *dependencias* de autenticación.
- **tests/**. Contiene las *pruebas* unitarias, de contrato, integración y E2E.
- **migrations/**. Contiene la configuración y las versiones de *Alembic*.
- **.github/workflows/**. Contiene el *pipeline* de integración continua y construcción de *Docker*.

ARCHIVOS IMPORTANTES

Configuración y aplicación:

- **pyproject.toml**. Define metadatos, dependencias, paquete y configuración de pytest, Ruff y mypy.
- **poetry.lock**. Conserva las versiones concretas de las dependencias.
- **src/orders_api/main.py**. Crea la aplicación *FastAPI*, registra las rutas y expone /health.
- **.env.example**. Documenta las variables de entorno necesarias sin incluir secretos reales.
- **.github/workflows/final-orders-ci.yml**. Automatiza calidad, pruebas, auditoría y construcción de Docker.

Capa domain/:

- **src/orders_api/domain/entities.py**. Define *Order*, *OrderItem* y las reglas de cambio de estado.
- **src/orders_api/domain/enums.py**. Define *categorías* de productos y *estados* de las órdenes.
- **src/orders_api/domain/catalog.py**. Contiene el *catálogo* fijo de seis productos.
- **src/orders_api/domain/errors.py**. Define los errores propios del dominio.

Capa application/:

- **src/orders_api/application/ports.py**. Define el puerto *OrderRepository*.

- **src/orders_api/application/schemas.py**. Define los *comandos* utilizados por los casos de uso.
- **src/orders_api/application/services.py**. Implementa la creación, consulta, listado, cambio de estado y eliminación de *órdenes*.

Capa api/:

- **src/orders_api/api/schemas.py**. Define los *modelos* de entrada y salida de la API.
- **src/orders_api/api/dependencies.py**. Configura la *API key*, las sesiones y el servicio de órdenes.
- **src/orders_api/api/routes.py**. Implementa los *endpoints* HTTP.

Capa infrastructure/:

- **src/orders_api/infrastructure/config.py**. Gestiona la configuración mediante *variables de entorno*.
- **src/orders_api/infrastructure/database.py**. Configura el engine, las sesiones y la clase base de *SQLAlchemy*.
- **src/orders_api/infrastructure/models.py**. Define el modelo persistente de la tabla orders.
- **src/orders_api/infrastructure/repositories.py**. Implementa el *adaptador* de persistencia con SQLAlchemy.

Capa migrations/:

- **alembic.ini**. Configura Alembic para ejecutar las migraciones.
- **migrations/env.py**. Conecta Alembic con los metadatos de SQLAlchemy.
- **migrations/versions/**. Contiene las *migraciones versionadas* de la base de datos.

Capa tests/:

- **tests/conftest.py**. Configura el cliente de pruebas y la base *SQLite temporal*.
- **tests/unit/**. Comprueba entidades, reglas y servicios.
- **tests/contract/**. Comprueba el contrato HTTP de la API.
- **tests/integration/**. Comprueba la persistencia con SQLAlchemy y SQLite.
- **tests/e2e/**. Comprueba los flujos completos de la API.

Docker:

- **Dockerfile**. Construye la imagen Docker multistage.
- **docker-entrypoint.sh**. Aplica las migraciones antes de iniciar Uvicorn.
- **.dockerignore**. Excluye archivos innecesarios del contexto de Docker.

Documentos:

- **README.md**. Documenta la instalación, arquitectura, API, pruebas, Docker y CI/CD.
- **docs/diagrams/**. Contiene los diagramas editables y sus imágenes exportadas.
- **docs/evidence/**. Contiene las capturas de evidencia del funcionamiento del proyecto.
- **Seguimiento-Curso_Python_v2.pdf**. Resume el contenido del curso y sus módulos. Segunda versión para mantener la primera como registro en el repositorio.

SEGUIMIENTO DEL PROYECTO

A diferencia de los laboratorios, para este proyecto fue mejor crear el **pyproject.toml** manualmente porque fue necesaria una configuración concreta de dependencias, paquetes, pytest, Ruff y mypy.

1. Se definió el alcance del servicio **Orders** y el catálogo fijo de **seis productos** para la tienda de figuras de acción. Los productos por nombre/categoría fueron:
 - ✓ Caballero / Figuras
 - ✓ Mago / Figuras
 - ✓ Robot / Figuras
 - ✓ Alienígena / Figuras
 - ✓ Accesorios / Complementos
 - ✓ Herramientas / Complementos
2. Se creó la **estructura** del proyecto con las capas de dominio, aplicación, infraestructura, API, migraciones y pruebas.
3. Se configuró Poetry mediante **pyproject.toml**, se seleccionó Python 3.12 y se generó **poetry.lock**.
4. Se implementó el **dominio** con entidades, estados, catálogo, reglas de negocio y errores específicos.
5. Se crearon los casos de uso y el puerto **OrderRepository**, manteniendo la aplicación independiente de la persistencia.
6. Se implementó la **persistencia** con SQLAlchemy y SQLite, junto con el **repositorio** y la migración inicial de Alembic.
7. Se desarrolló la **API FastAPI** con esquemas, endpoints de órdenes, healthcheck y autenticación mediante API key.

8. Se añadieron las **operaciones** para crear, consultar, listar, actualizar el estado y eliminar órdenes pendientes.
9. Se implementaron **pruebas** unitarias, de contrato, integración y E2E con una base SQLite temporal.
10. Se configuraron Ruff, mypy, pytest, cobertura y pip-audit, corrigiendo errores de tipado, lint y compatibilidad.
11. Se construyó el **Dockerfile** multistage y el **script de entrada** para ejecutar migraciones antes de iniciar la API.
12. Se corrigieron diferencias de versión de Poetry, finales de línea del script Docker y configuración de la base SQLite en pruebas.
13. Se validaron localmente la aplicación, las pruebas, la cobertura, la auditoría de dependencias, Docker y los endpoints.
14. Se creó el workflow **final-orders-ci.yml** para automatizar calidad, pruebas, auditoría, empaquetado y construcción de Docker.
15. Se configuró **.gitattributes** para conservar finales de línea LF en scripts y evitar errores de ejecución de **docker-entrypoint.sh** dentro del contenedor.
16. Se subieron los cambios a la rama **proyecto-final**, se revisaron los checks y se fusionaron mediante un **Pull Request**.
17. Se prepararon el **README**, los diagramas y las capturas de evidencia del proyecto.

RESULTADOS DEL PROYECTO

DIAGRAMAS

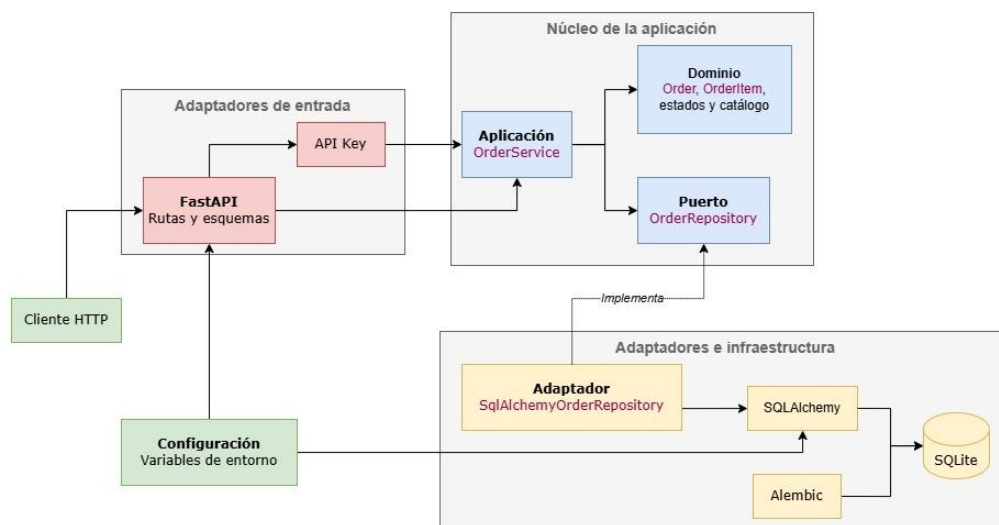


Diagrama 1. Arquitectura.

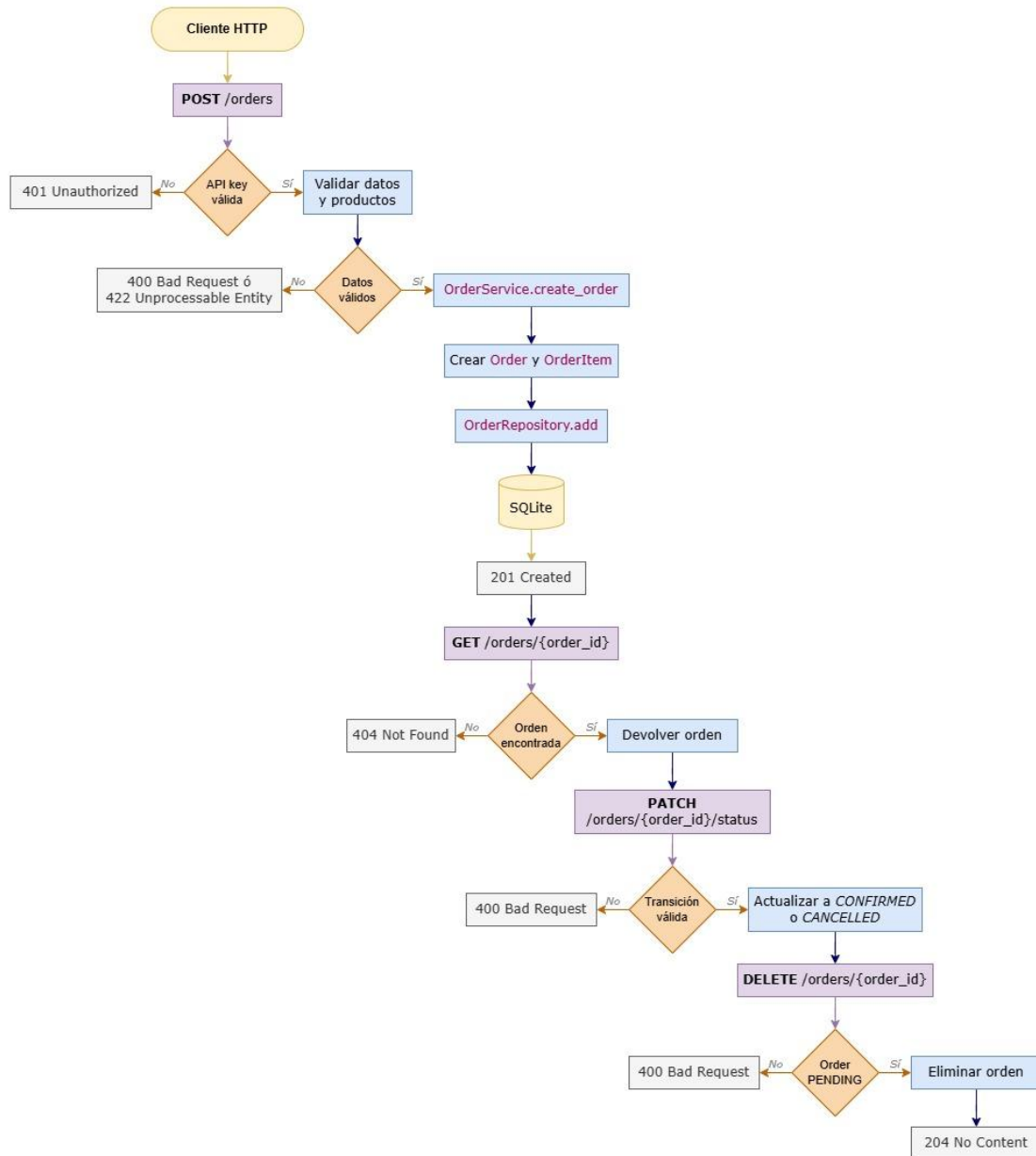


Diagrama 2. Flujo de creación, actualización y eliminación de una orden.

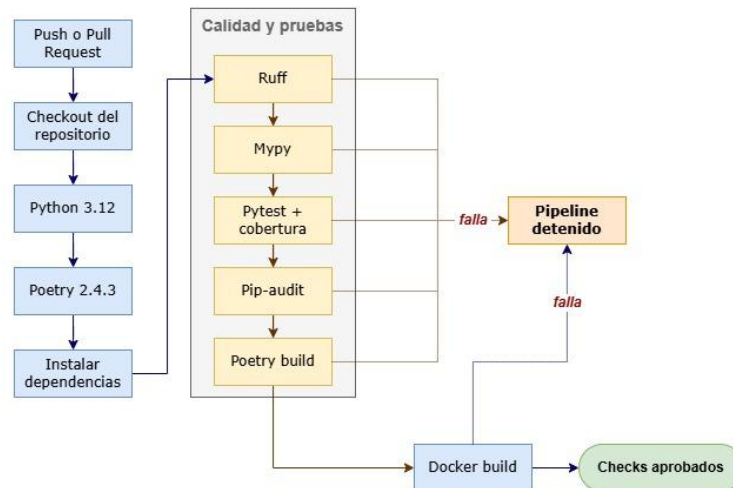


Diagrama 3. Pipeline CI/CD configurado para GitHub Actions.

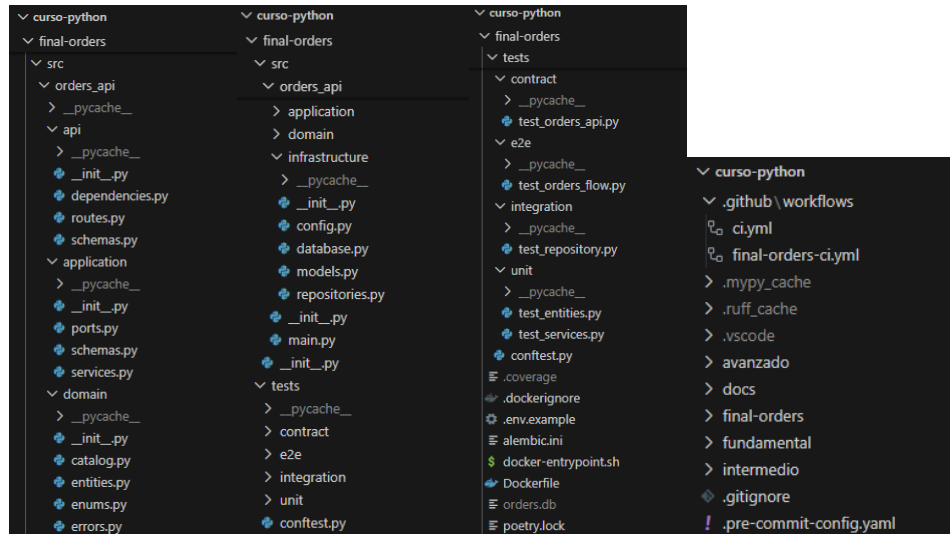
PRUEBAS REALIZADAS

Todas las siguientes pruebas pasaron, sin vulnerabilidades y con una cobertura aproximada del 93%. Así mismo, Docker construyó y ejecutó correctamente la aplicación.

- **Pruebas unitarias.** Valida entidades, productos, estados, reglas de negocio y servicios sin depender de la base de datos.
- **Pruebas de contrato.** Comprueba esquemas, endpoints, API key, códigos HTTP y respuestas de la API.
- **Pruebas de integración.** Verifica la *persistencia de órdenes* mediante SQLAlchemy, SQLite y el repositorio.
- **Pruebas E2E.** Comprueba los *flujos completos* de creación, consulta, cambio de estado y eliminación.
- **Pruebas de cobertura.** Mide el *porcentaje* de código ejecutado por la suite de pruebas. Se
- **Ruff.** Detecta errores de formato, estilo, imports y posibles problemas del código.
- **Mypy.** Comprueba la consistencia estática de los tipos.
- **Pip-audit.** Revisa las dependencias instaladas frente a *vulnerabilidades* conocidas.
- **Migraciones Alembic.** Verifica la creación y actualización del esquema de la base de datos.
- **Prueba de Docker.** Comprueba la construcción y ejecución de la *imagen multistage*.
- **Prueba de API en contenedor.** Verifica /health y los endpoints protegidos dentro del contenedor.
- **Prueba de CI/CD.** Confirma que *GitHub Actions* ejecuta calidad, pruebas, auditoría y build correctamente.

CAPTURAS

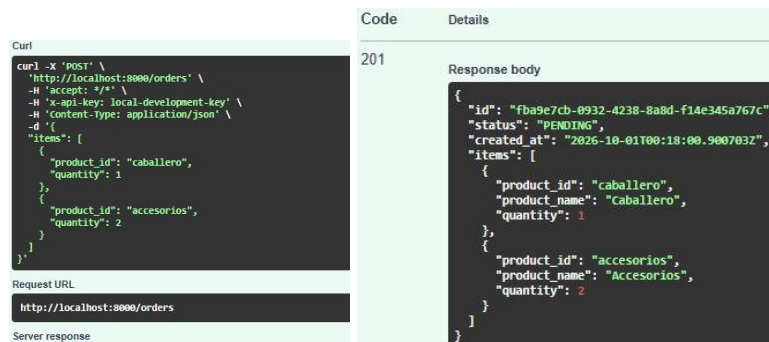
Estructura del Proyecto. Diferentes vistas al interior de la estructura desde VS Code.



Swagger con los endpoints. Vista desde navegador Web.



Flujo de creación y seguridad. Creación de una orden mediante POST.



Flujo de estados y eliminación. Actualización del estado de una orden a “CONFIRMED” para intentar su eliminación. Creación de una nueva orden con estado “PENDING” para poder eliminarla posteriormente.

```

Curl
curl -X 'PATCH' \
  'http://localhost:8000/orders/fba9e7cb-0932-4238-8a8d-f14e345a767c/status' \
  -H 'accept: */*' \
  -H 'x-api-key: local-development-key' \
  -H 'Content-Type: application/json' \
  -d '{
    "status": "CONFIRMED"
  }'

Request URL
http://localhost:8000/orders/fba9e7cb-0932-4238-8a8d-f14e345a767c/status

Server response
Code    Details
200

Response body
{
  "id": "fba9e7cb-0932-4238-8a8d-f14e345a767c",
  "status": "CONFIRMED",
  "created_at": "2026-10-01T00:18:00.900703Z",
  "items": [
    {
      "product_id": "Caballero",
      "product_name": "Caballero",
      "quantity": 1
    },
    {
      "product_id": "Accesorios",
      "product_name": "Accesorios",
      "quantity": 2
    }
  ]
}

```

```

Curl
curl -X 'DELETE' \
  'http://localhost:8000/orders/fba9e7cb-0932-4238-8a8d-f14e345a767c' \
  -H 'accept: */*' \
  -H 'x-api-key: local-development-key'

Request URL
http://localhost:8000/orders/fba9e7cb-0932-4238-8a8d-f14e345a767c

Server response
Code    Details
400     Error: Bad Request
Undocumented

Response body
{
  "detail": "Solo se pueden eliminar órdenes pendientes."
}

Response headers
content-length: 37
content-type: application/json
date: Thu, 01 Oct 2026 00:22:32 GMT
server: uvicorn

```

Creación de una nueva orden, y fallo al eliminar una orden confirmada.

```

Curl
curl -X 'DELETE' \
  'http://localhost:8000/orders/abc59616-1735-43fb-afaf-fea1edbe3db8' \
  -H 'accept: */*' \
  -H 'x-api-key: local-development-key'

Request URL
http://localhost:8000/orders/abc59616-1735-43fb-afaf-fea1edbe3db8

Server response
Code    Details
204

Response headers
content-type: application/json
date: Thu, 01 Oct 2026 00:23:42 GMT
server: uvicorn

```

Eliminación exitosa de una orden sin confirmar.

Pruebas y cobertura. Con un 93%.

```
$ poetry run pytest --cov=src --cov-report=term-missing
```

Name	Stmts	Miss	Cover	Missing
src__init__.py	0	0	100%	
src\orders_api__init__.py	0	0	100%	
src\orders_api\api__init__.py	0	0	100%	
src\orders_api\api\dependencies.py	14	0	100%	
src\orders_api\api\routes.py	46	7	85%	73-74, 109-115, 132
src\orders_api\api\schemas.py	22	0	100%	
src\orders_api\application__init__.py	0	0	100%	
src\orders_api\application\ports.py	5	0	100%	
src\orders_api\application\schemas.py	10	0	100%	
src\orders_api\application\services.py	33	3	91%	38-41
src\orders_api\domain__init__.py	0	0	100%	
src\orders_api\domain\catalog.py	13	0	100%	
src\orders_api\domain\entities.py	39	2	95%	35, 59
src\orders_api\domain\enums.py	8	0	100%	
src\orders_api\domain\errors.py	4	0	100%	
src\orders_api\infrastructure__init__.py	0	0	100%	
src\orders_api\infrastructure\config.py	10	0	100%	
src\orders_api\infrastructure\database.py	15	2	87%	35-36
src\orders_api\infrastructure\models.py	10	0	100%	
src\orders_api\infrastructure\repositories.py	53	4	92%	35-39, 45, 56
src\orders_api\main.py	7	1	86%	14
TOTAL	289	19	93%	

20 passed, 1 warning in 0.42s

Ruff, mypy y pip-audit. Validaciones de calidad desde VS Code.

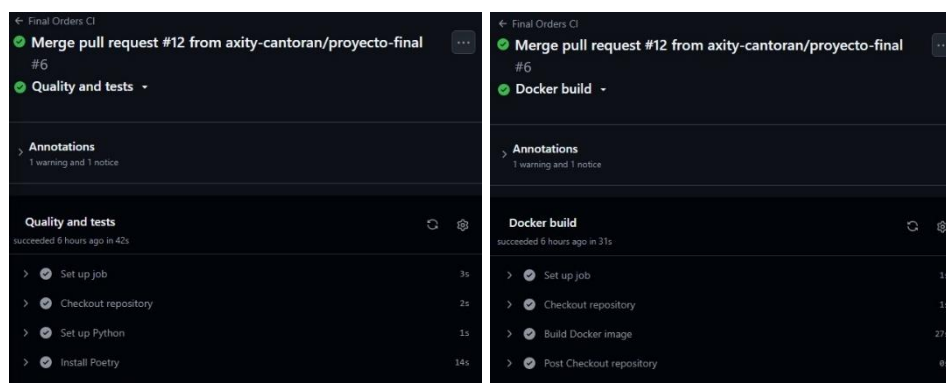
```
$ poetry run ruff check src tests && \
> poetry run mypy src/orders_api && \
> poetry run pip-audit
All checks passed!
Success: no issues found in 20 source files
No known vulnerabilities found
Name      Skip Reason
-----
final-orders Dependency not found on PyPI and could not be audited: final-orders (0.1.0)
```

Docker funcionando. Construcción de la imagen y revisión de /health.

```
$ docker build --no-cache -t final-orders:local .
[+] Building 24.5s (22/22) FINISHED          docker:desktop-linux
=> [internal] load build definition from Dockerfile      0.0s
=> => transferring dockerfile: 1.17kB                    0.0s
=> [internal] load metadata for docker.io/library/python:3.12-slim 0.0s
=> [internal] load .dockerignore                        0.0s
=> => transferring context: 171B                          0.0s
=> CACHED [builder 1/9] FROM docker.io/library/python:3.12-slim@sha256: 0.0s
=> => resolve docker.io/library/python:3.12-slim@sha256:f77ac9e44ae96e 0.0s
=> [internal] load build context                        0.0s
=> => transferring context: 3.98kB                        0.0s
=> CACHED [builder 2/9] WORKDIR /build                  0.0s
=> [builder 3/9] RUN pip install --no-cache-dir "poetry==2.4.3"      9.3s
=> [runtime 2/9] RUN useradd --create-home --shell /usr/sbin/n 0.4s
=> [runtime 3/9] WORKDIR /app                            0.1s
=> [builder 4/9] COPY pyproject.toml poetry.lock ./        0.1s
=> [builder 5/9] RUN poetry install --only main --no-root      3.9s
=> [builder 6/9] COPY src ./src                          0.1s
=> [builder 7/9] COPY alembic.ini ./                      0.1s
=> [builder 8/9] COPY migrations ./migrations             0.1s
=> [builder 9/9] RUN poetry install --only main            1.3s
=> [runtime 4/9] COPY --from=builder /build/.venv /opt/venv 0.2s
=> [runtime 5/9] COPY --from=builder /build/alembic.ini ./ 0.1s
=> [runtime 6/9] COPY --from=builder /build/migrations ./migrations 0.1s
=> [runtime 7/9] COPY src ./src                          0.1s
=> [runtime 8/9] COPY docker-entrypoint.sh ./            0.1s
=> [runtime 9/9] RUN chmod +x docker-entrypoint.sh && chown -R app 5.5s
=> exporting to image                                    3.4s
=> => exporting layers                                    2.3s
=> => exporting manifest sha256:9b3238b5e5952524cc94ab5c7261369ed60381 0.0s
=> => exporting config sha256:659be4ba36949efd5b087b4a69bfb86d71d24bb0 0.0s

$ curl http://localhost:8000/health
{"status":"ok"}
```

GitHub Actions y Pull request. Actions aprobado y Pull request funcionando en github.com.



Procesos de calidad y pruebas, y construcción de Docker aprobados.



Rama fucionada mediante Pull Request.

RESULTADOS FINALES

El proyecto implementa correctamente un servicio de órdenes para una tienda de figuras de acción con un catálogo fijo de seis productos. Permite la creación de órdenes con uno o varios productos, consulta, confirmación, cancelación y eliminación de órdenes. Además, la API FastAPI se encuentra protegida mediante API key.

Otros resultados obtenidos:

- ✓ Documentación automática mediante Swagger/OpenAPI.
- ✓ Persistencia en SQLite con SQLAlchemy.
- ✓ Migraciones funcionales con Alembic.
- ✓ Auditoría final de dependencias sin vulnerabilidades.
- ✓ Imagen Docker multistage construida y ejecutada con usuario no root.
- ✓ Migraciones ejecutadas automáticamente al iniciar el contenedor.
- ✓ Pipeline CI configurado con GitHub Actions.
- ✓ Construcción automatizada del paquete y de la imagen Docker.